Gestion des bases et des fichiers

2.1 Création de la base de données TechShop_DB

La base de données **TechShop_DB** a été créée afin de stocker les informations liées aux clients, aux produits et aux commandes de l'entreprise.
Elle repose sur une architecture multi-fichier comprenant :

- un fichier de données principal (MDF),
- un fichier journal de transactions (LDF),
- une croissance définie en **mégaoctets** afin de maîtriser l'évolution de l'espace disque.

Les tailles initiales et les paramètres de croissance ont été configurés de manière cohérente pour répondre aux besoins de l'application.

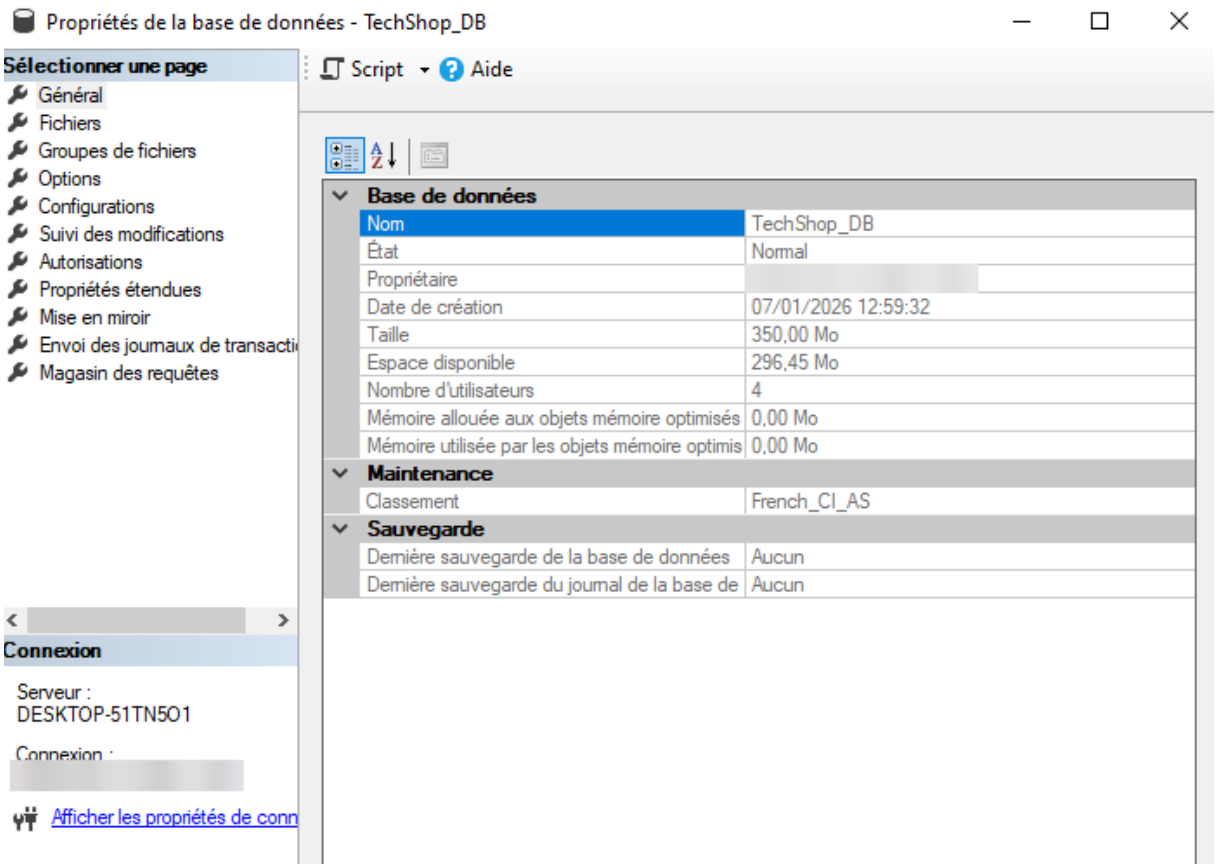
2.2 Organisation des fichiers et filegroups

Afin d'améliorer l'organisation des données et d'anticiper la volumétrie, un **filegroup secondaire nommé FG_COMMANDES** a été créé.
Ce filegroup est destiné à accueillir les tables volumineuses, notamment celles liées aux commandes.

La base TechShop_DB contient donc :

- le filegroup **PRIMARY**, utilisé pour les tables CLIENTS et PRODUITS,
- le filegroup **FG_COMMANDES**, utilisé pour la table COMMANDES.

Cette séparation permet de faciliter la maintenance, les sauvegardes et les optimisations futures.



2.3 Création des tables principales

Trois tables principales ont été créées pour répondre aux besoins fonctionnels de l'entreprise :

Table CLIENTS

- ID (clé primaire)
- Nom
- Email
- Ville

Table PRODUITS

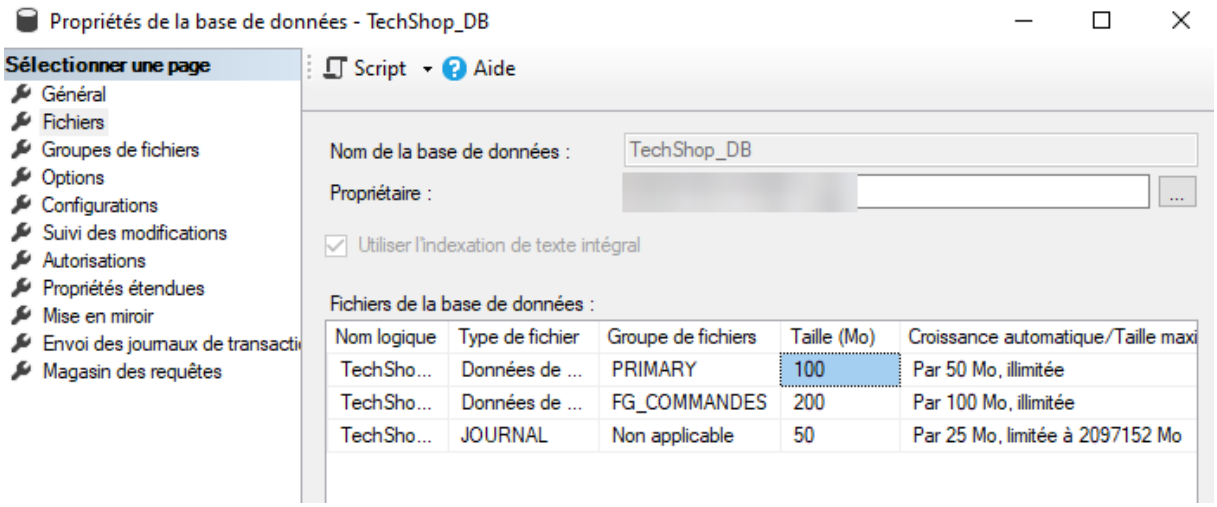
- ID (clé primaire)
- Libellé

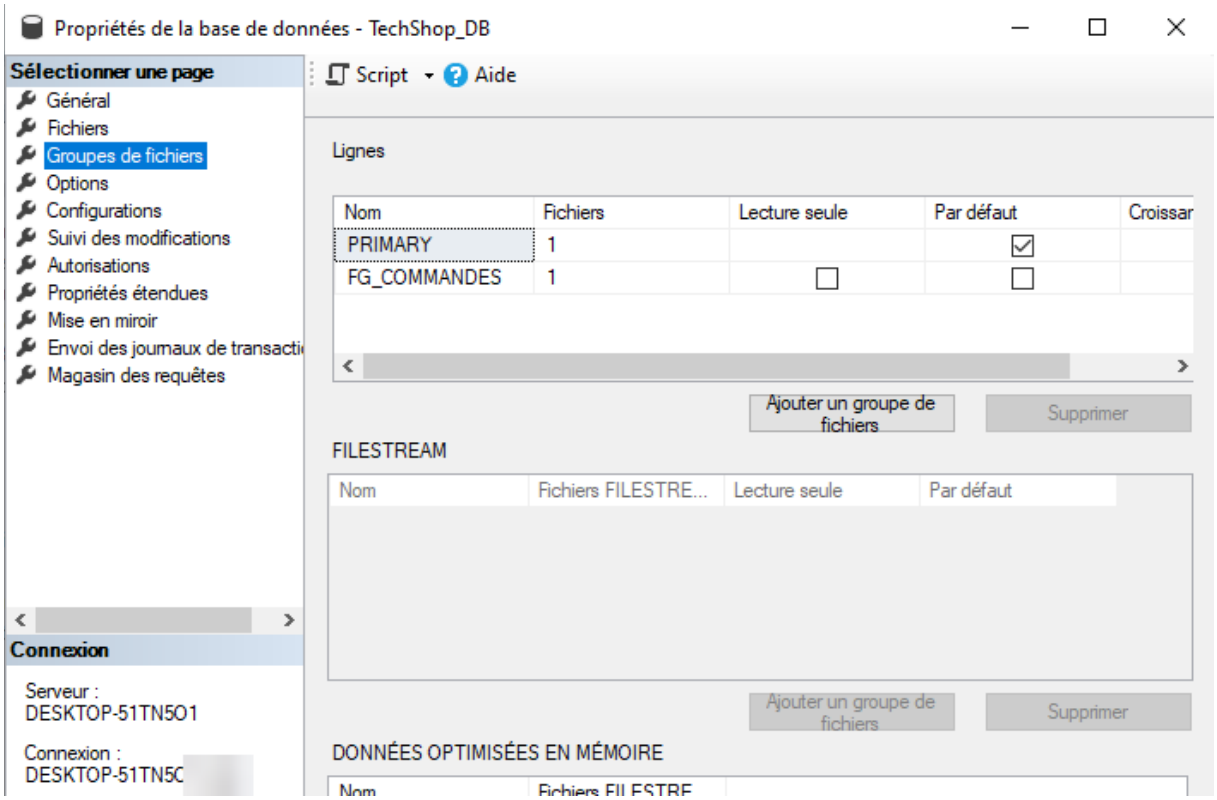
- Prix
- Stock

Table COMMANDES

- ID (clé primaire)
- Date
- ClientID (clé étrangère vers CLIENTS)
- ProduitID (clé étrangère vers PRODUITS)
- Quantité

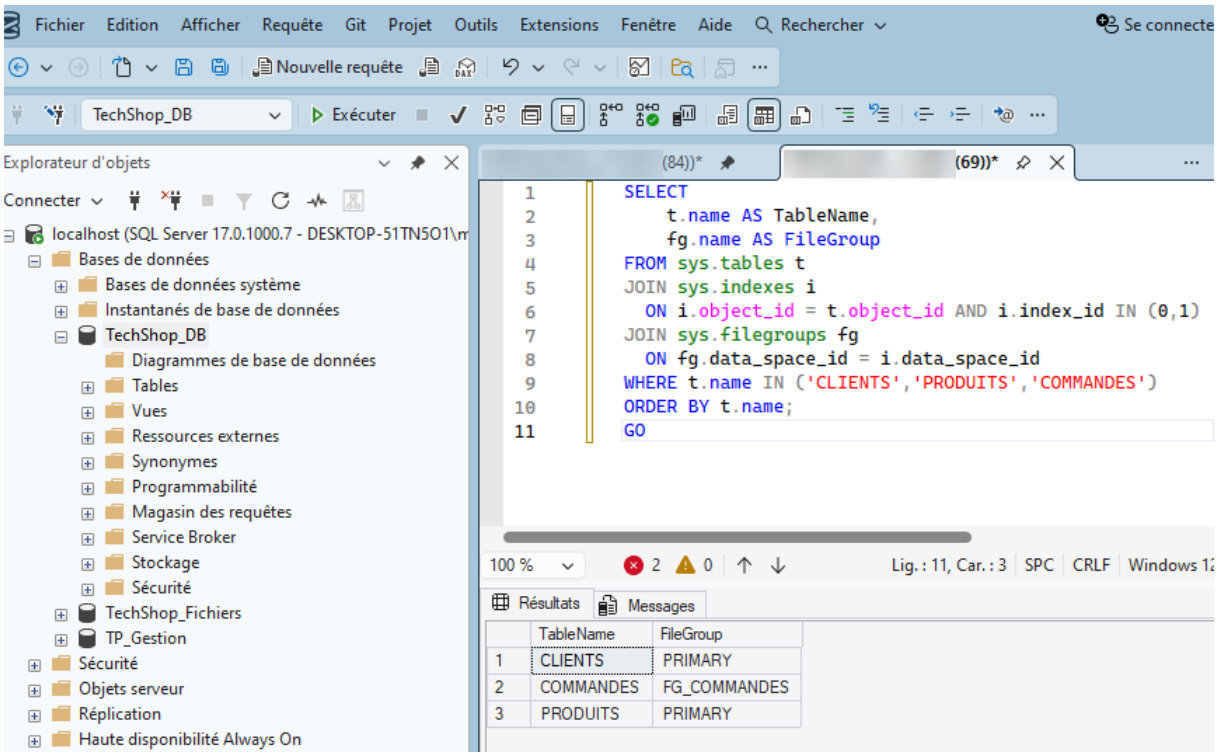
La table COMMANDES a été explicitement créée sur le filegroup **FG_COMMANDES**, tandis que les tables CLIENTS et PRODUITS sont stockées dans le filegroup PRIMARY.





2.4 Insertion de données de test

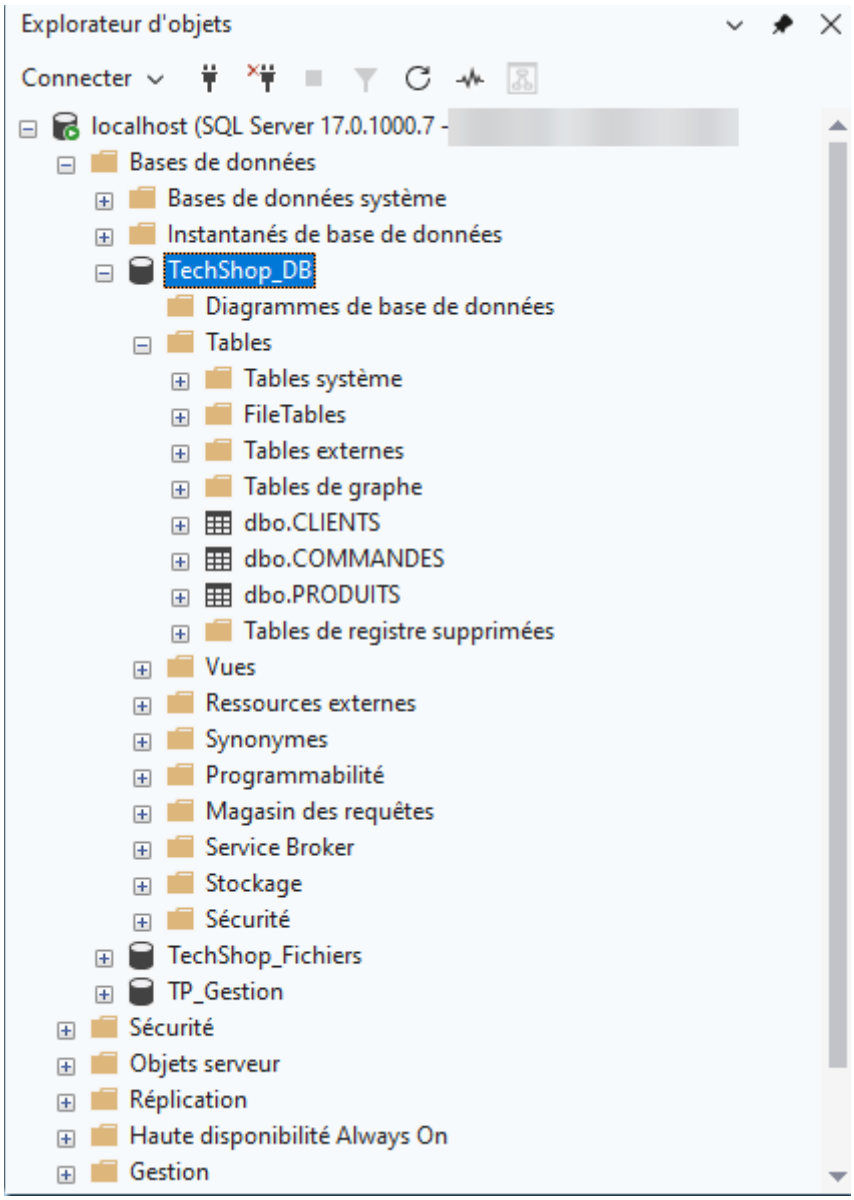
Des données de test ont été insérées dans les tables CLIENTS, PRODUITS et COMMANDES afin de valider le bon fonctionnement de la base et des relations entre les tables.
Ces insertions confirment l'intégrité référentielle et la cohérence des données.



2.5 Vérification de la répartition des tables dans les filegroups

La répartition physique des tables a été vérifiée à l'aide des vues système de SQL Server.
Cette vérification confirme que :

- CLIENTS est stockée dans PRIMARY,
- PRODUITS est stockée dans PRIMARY,
- COMMANDES est stockée dans FG_COMMANDES.

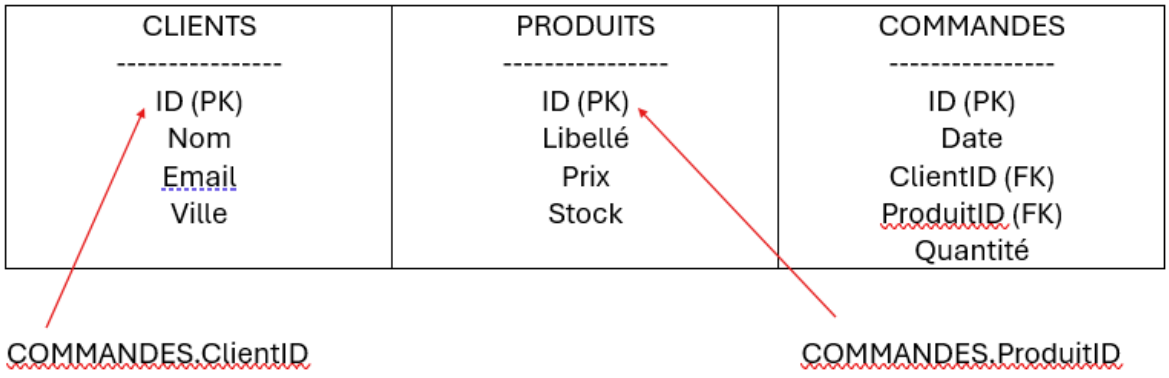


2.6 Schéma logique simplifié de la base TechShop_DB

Le schéma logique simplifié ci-dessous présente les entités principales de la base et leurs relations :

- CLIENTS est lié à COMMANDES par la clé étrangère ClientID,
- PRODUITS est lié à COMMANDES par la clé étrangère ProduitID.

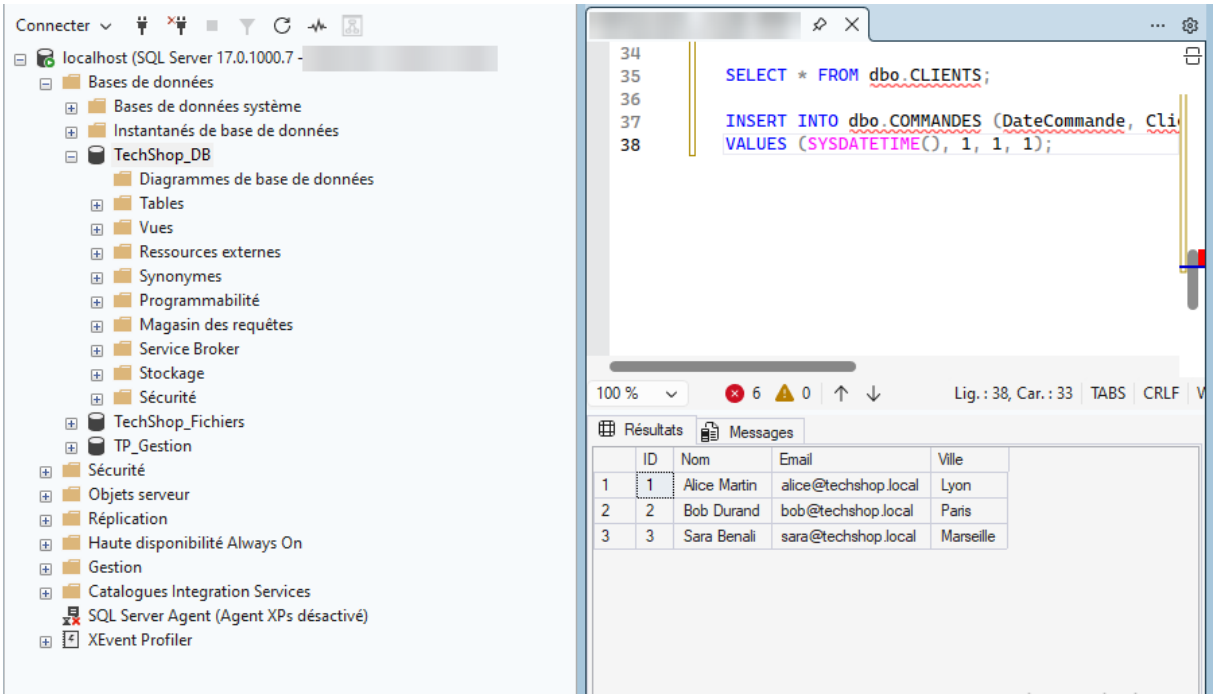
Ce schéma permet de visualiser clairement la structure de la base et les dépendances entre les données.



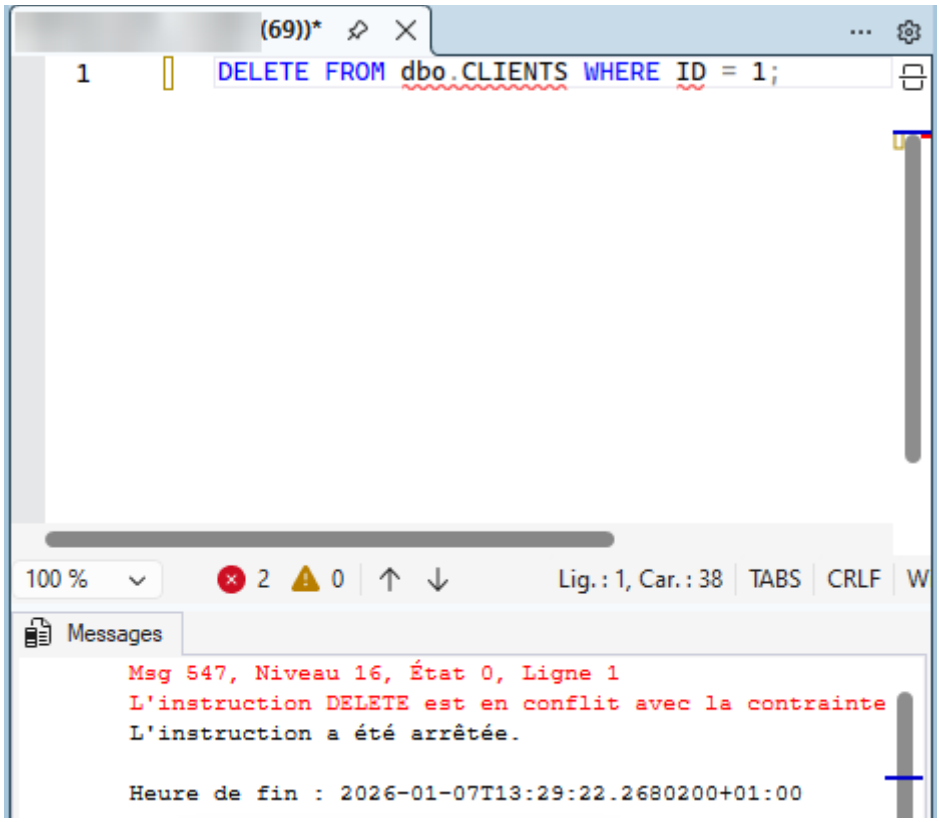
Modification des données après sauvegarde complète

Après la sauvegarde complète, de nouvelles données ont été insérées dans la table **COMMANDES**.
Ces données représentent des transactions réalisées après le point de sauvegarde FULL.

Cette étape permet de démontrer l'intérêt des sauvegardes différentielles.



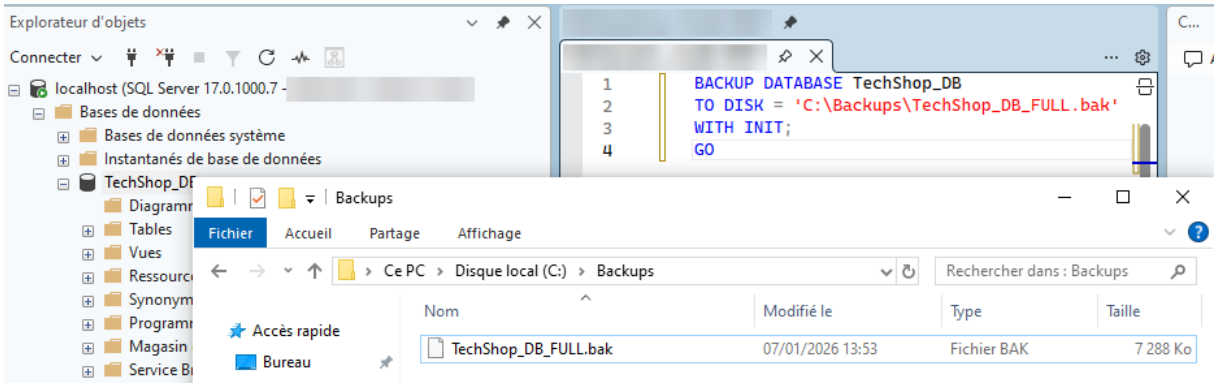
Qassim & Marouane



Mise en place de la stratégie de sauvegarde

Afin de garantir la protection des données de l'entreprise et de limiter les pertes en cas d'incident, une stratégie de sauvegarde a été mise en place sur la base **TechShop_DB**.

Une **sauvegarde complète (FULL)** a été réalisée afin de constituer un point de référence stable pour la restauration.

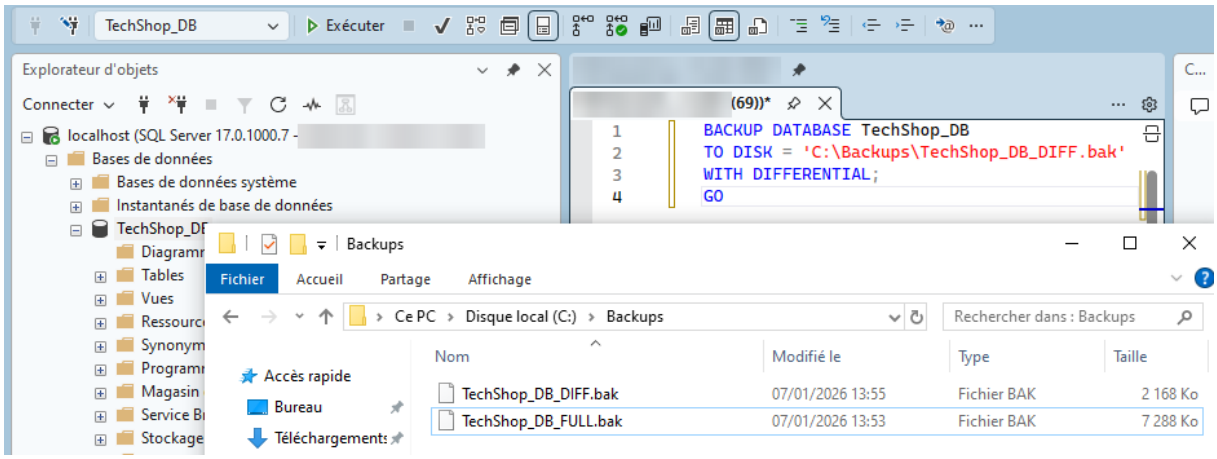


Sauvegarde différentielle

Une **sauvegarde différentielle (DIFF)** a ensuite été effectuée afin d'enregistrer uniquement les modifications intervenues depuis la dernière sauvegarde complète.

Les fichiers de sauvegarde suivants sont présents sur le serveur :

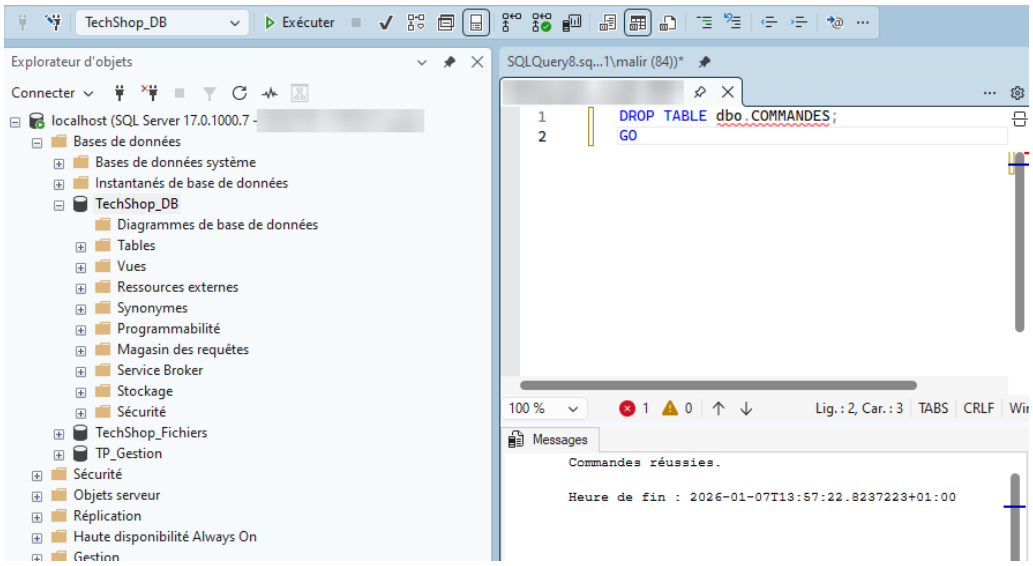
- TechShop_DB_FULL.bak
- TechShop_DB_DIFF.bak



Simulation d'incident

Un incident a été simulé par la suppression volontaire de la table **COMMANDES**.
Cette action représente une perte critique de données nécessitant une restauration.

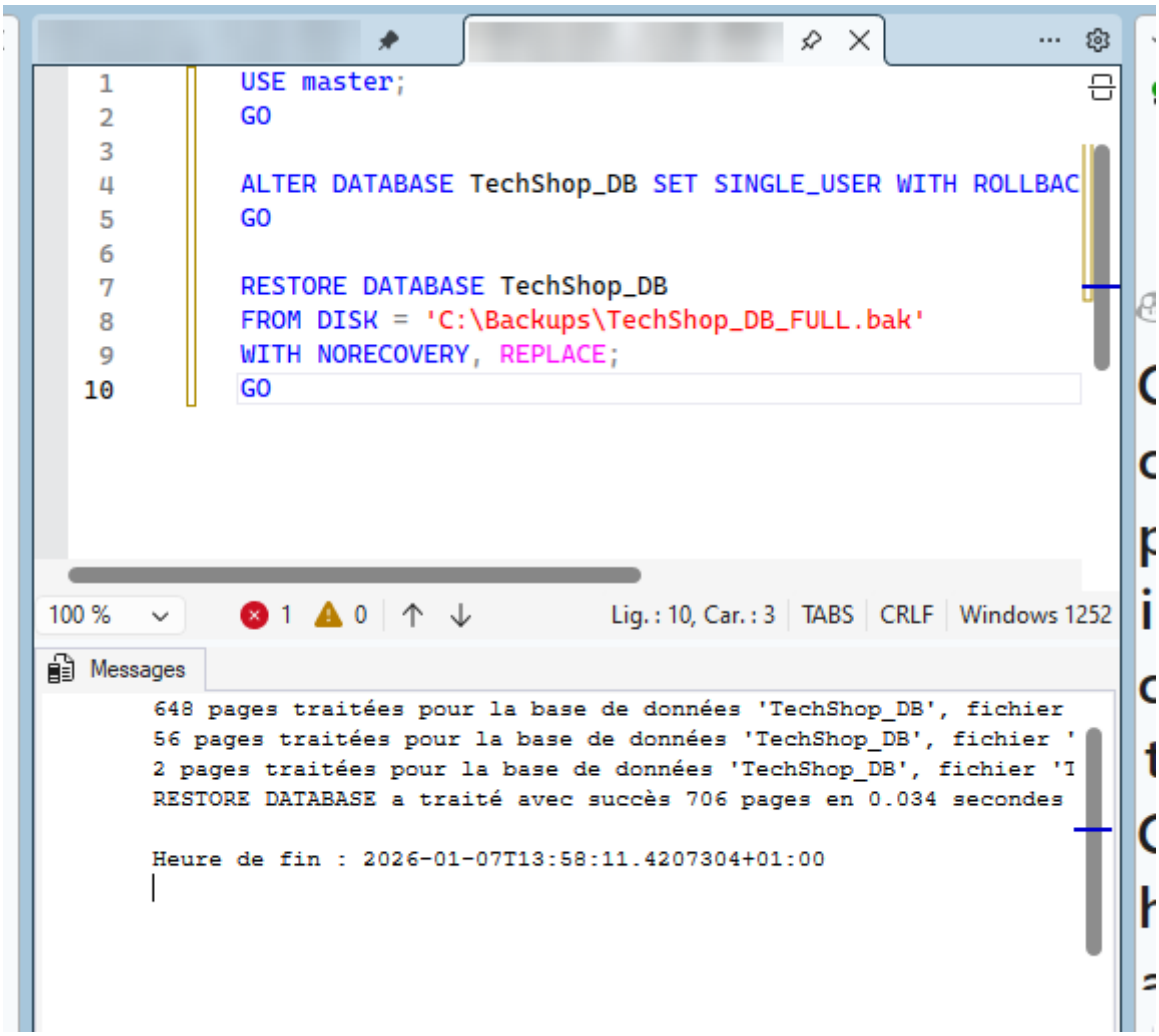
Qassim & Marouane



Restauration de la sauvegarde complète (NORECOVERY)

La restauration a été initiée à partir de la sauvegarde complète avec l'option **NORECOVERY**.
Cette option place la base dans l'état **RESTORING**, la rendant temporairement inaccessible afin de permettre l'application d'une sauvegarde complémentaire.

L'état de la base a été vérifié à l'aide de la vue système sys.databases.

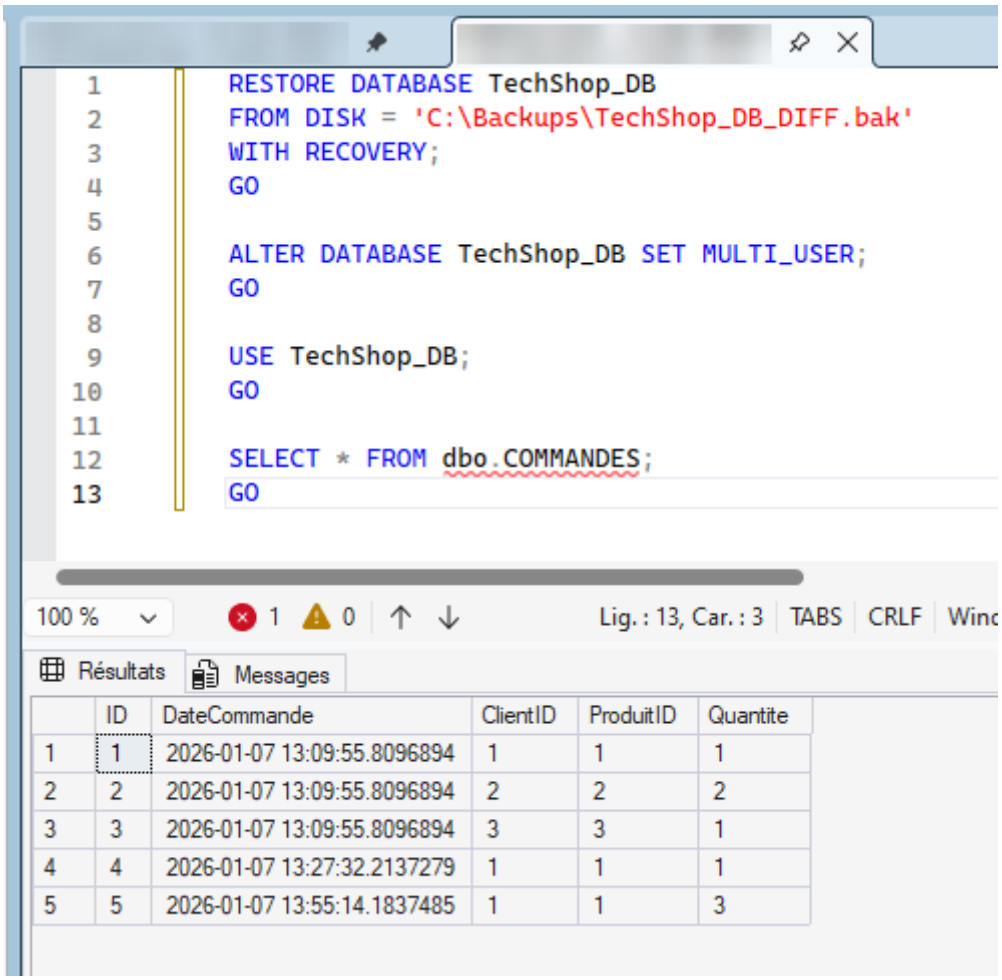


Restauration de la sauvegarde différentielle (RECOVERY)

La restauration de la sauvegarde différentielle a ensuite été effectuée avec l'option **RECOVERY**, permettant à la base de repasser à l'état **ONLINE** et de redevenir accessible aux utilisateurs.

Les données ajoutées après la sauvegarde complète ont été récupérées avec succès.

Qassim & Marouane



```
RESTORE DATABASE TechShop_DB
FROM DISK = 'C:\Backups\TechShop_DB_DIFF.bak'
WITH RECOVERY;
GO

ALTER DATABASE TechShop_DB SET MULTI_USER;
GO

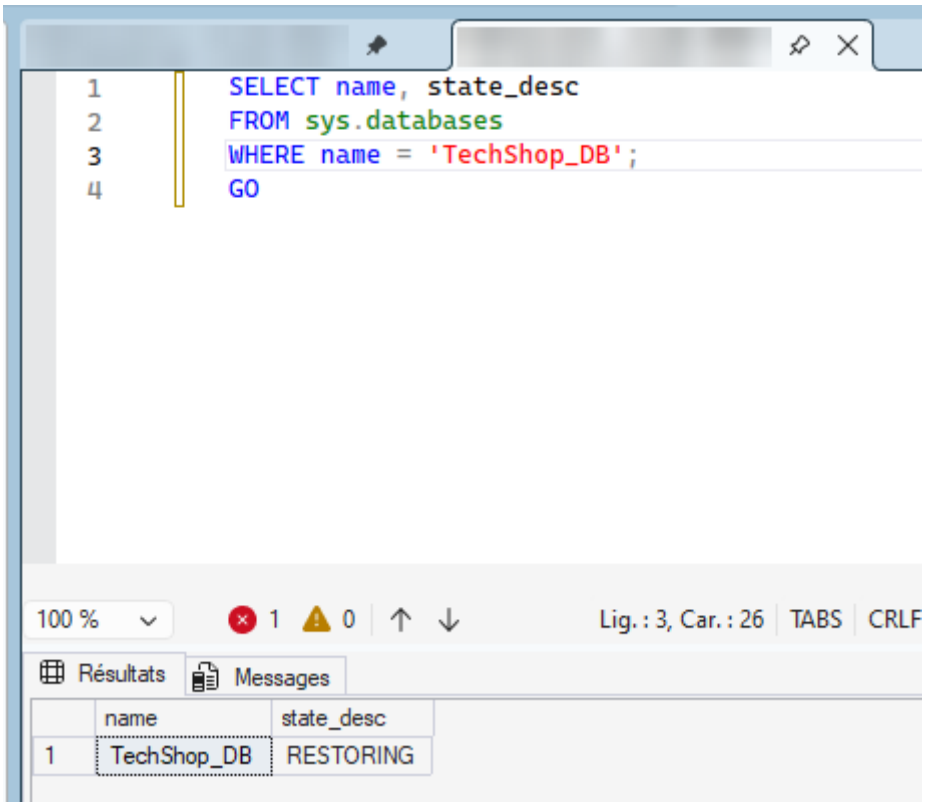
USE TechShop_DB;
GO

SELECT * FROM dbo.COMMANDES;
GO
```

100 % | 1 | 0 | Lig. : 13, Car. : 3 | TABS | CRLF | Winc

Résultats | Messages

	ID	DateCommande	ClientID	ProduitID	Quantite
1	1	2026-01-07 13:09:55.8096894	1	1	1
2	2	2026-01-07 13:09:55.8096894	2	2	2
3	3	2026-01-07 13:09:55.8096894	3	3	1
4	4	2026-01-07 13:27:32.2137279	1	1	1
5	5	2026-01-07 13:55:14.1837485	1	1	3

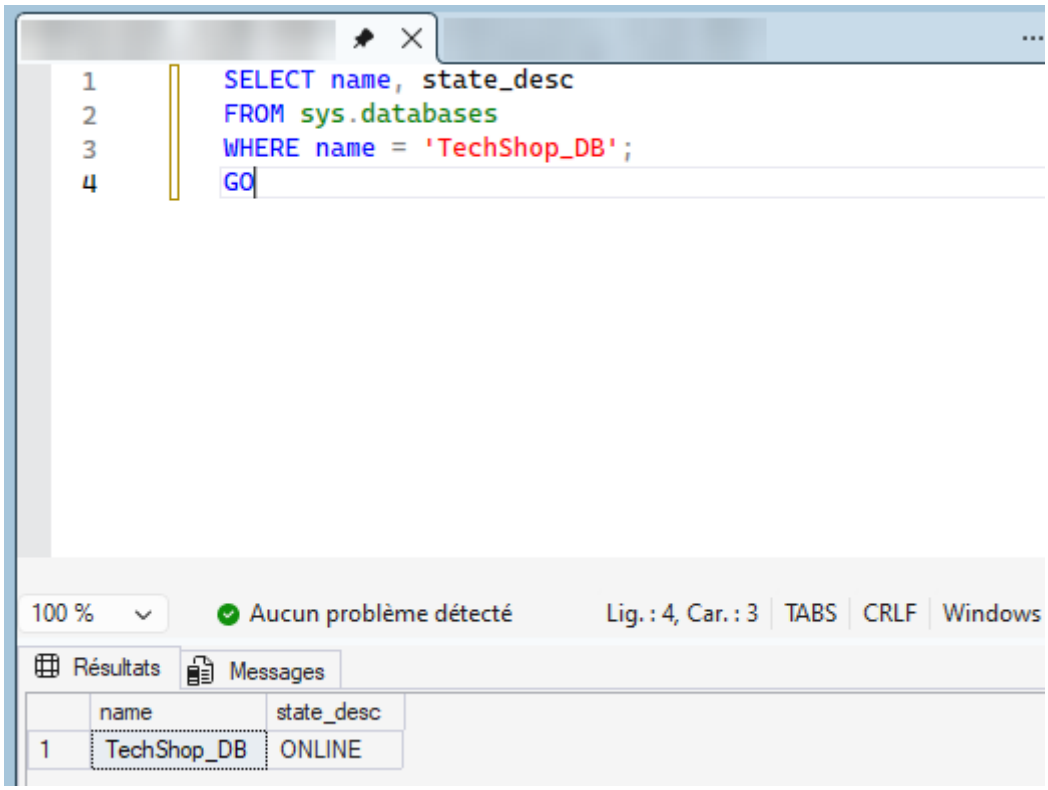


```
SELECT name, state_desc
FROM sys.databases
WHERE name = 'TechShop_DB';
GO
```

100 % | 1 | 0 | Lig. : 3, Car. : 26 | TABS | CRLF

Résultats | Messages

	name	state_desc
1	TechShop_DB	RESTORING

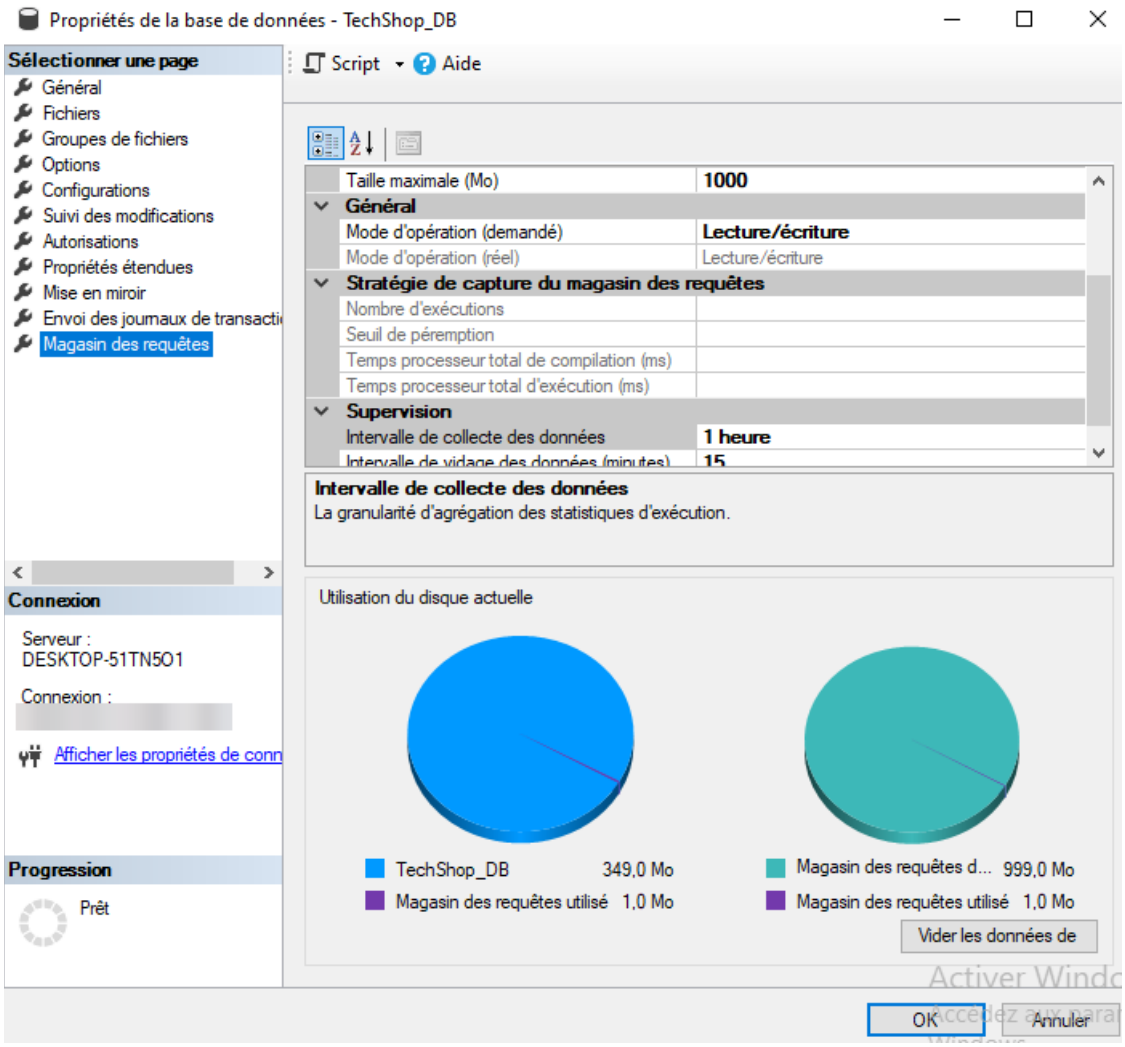


Supervision et performance

5.1 Activation du Magasin des requêtes (Query Store)

Le **Magasin des requêtes (Query Store)** a été activé sur la base TechShop_DB afin de collecter les statistiques d'exécution et d'analyser les performances des requêtes.

Le mode de fonctionnement est configuré en **lecture/écriture**, permettant l'enregistrement des plans d'exécution et des temps de traitement.

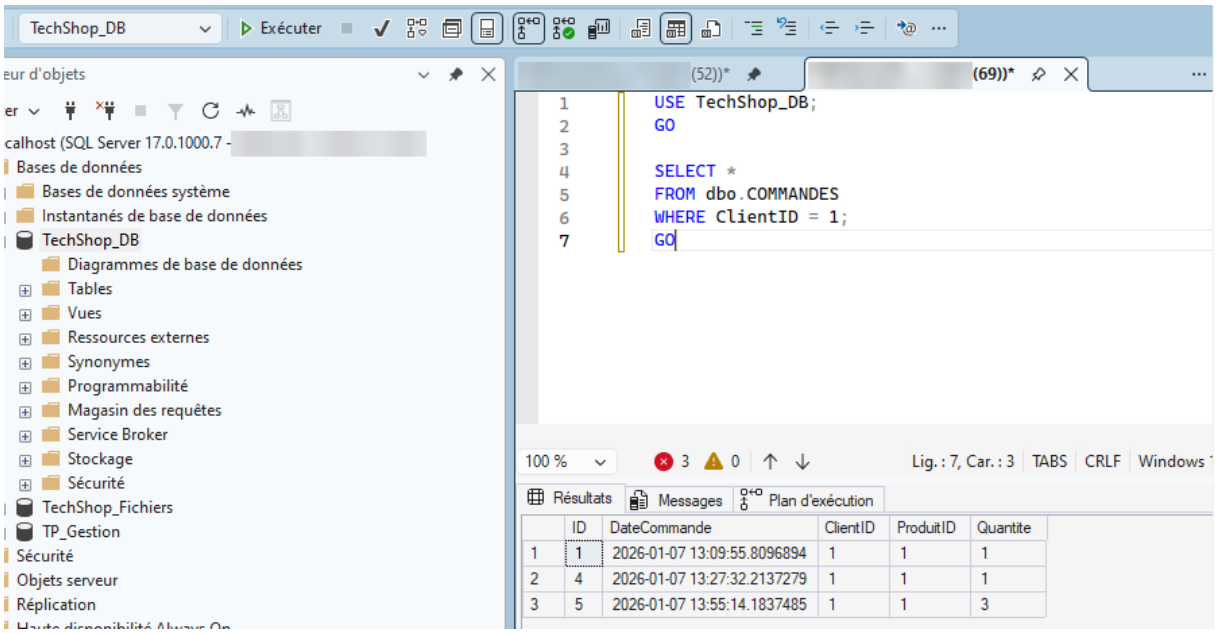


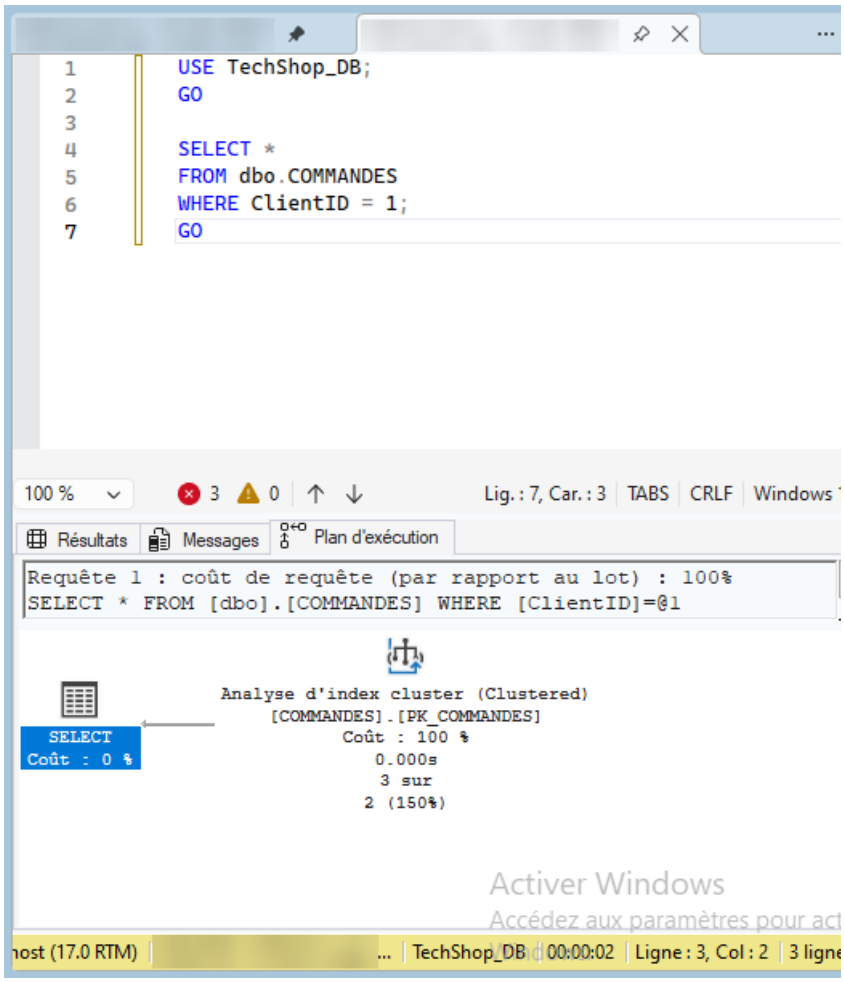
Analyse des requêtes sans index

Des requêtes de recherche ont été exécutées sur la table **COMMANDES**, notamment sur la colonne **ClientID**, sans index spécifique.

L'analyse du plan d'exécution montre une lecture complète de la table (Table Scan), indiquant un coût élevé pour les recherches fréquentes.

Qassim & Marouane

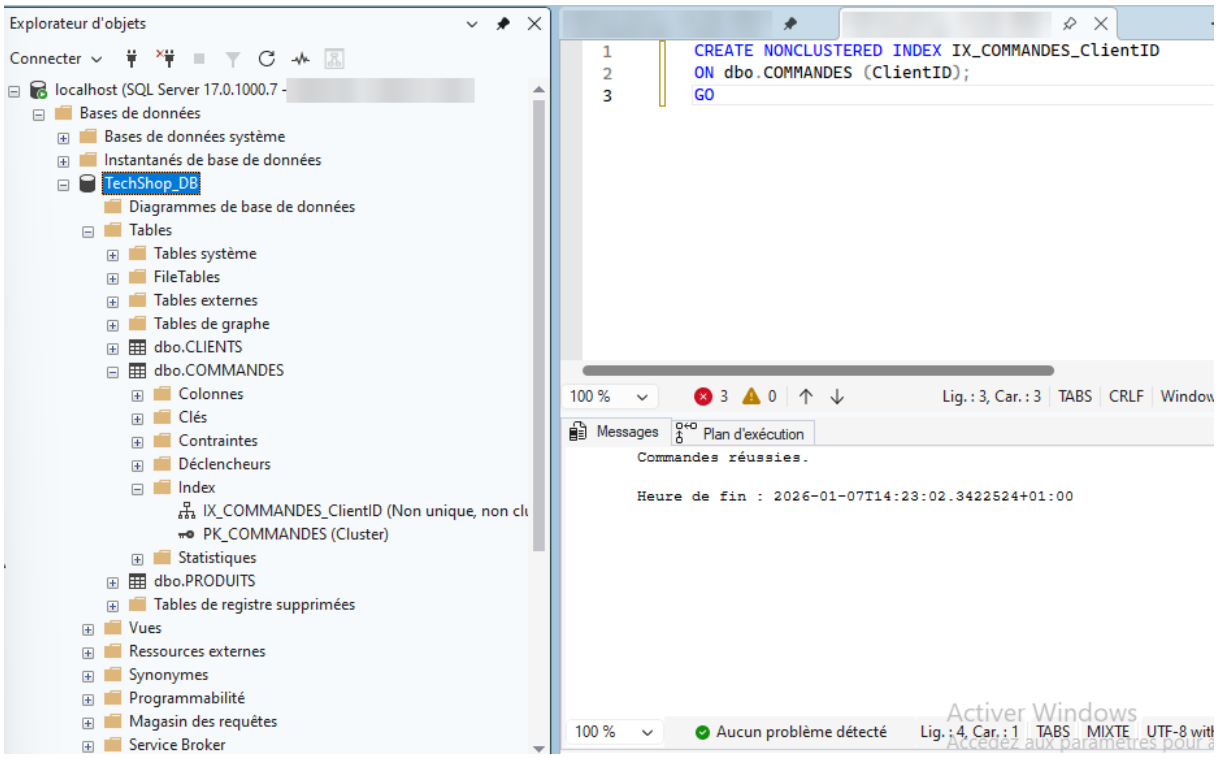




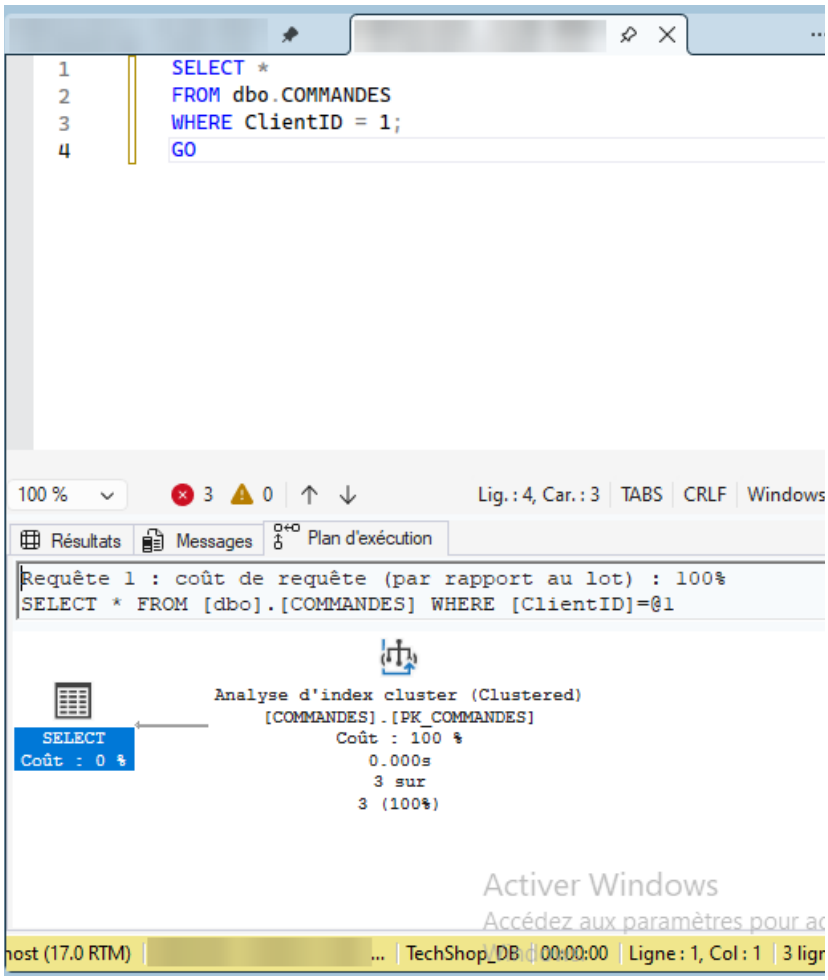
Optimisation par création d’index

Afin d’améliorer les performances, un **index non cluster** a été créé sur la colonne **ClientID**, qui est régulièrement utilisée dans les filtres de recherche.

Après la création de l’index, les requêtes ont été rejouées et les plans d’exécution analysés.



Qassim & Marouane



Wait stats

Qassim & Marouane

```
1  SELECT *
2  FROM sys.dm_os_wait_stats
3  ORDER BY wait_time_ms DESC;
4  GO
```

100 % | 3 | 0 | Lig. : 4, Car. : 3 | TABS | CRLF | Windows

Résultats | Messages | Plan d'exécution

		waiting_tasks_count	wait_time_ms	max_wait_time_ms	signal_wait_
1	IER	5586593	249649464	4735649	845851
2		66605	65713707	3518245	43207
3	SEMAPHORE	318	50136795	5906711	689
4		572	33852976	68323	2416
5	T	332	16998665	158871	0
6	MGR_IOCOMPLETION	33088	16946865	4268	12715
7		4540	16946525	7294	16931897
8		152570	16946170	4238	1448
9		121462	16945981	4269	877
10	LOCK_SEARCH	3383	16945324	6156	16945324

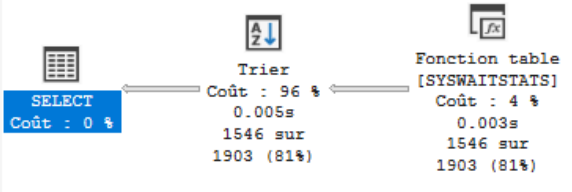
Activer Windows
Accédez aux paramètres pour act


```
1  SELECT *
2  FROM sys.dm_os_wait_stats
3  ORDER BY wait_time_ms DESC;
4  GO
```

100 % | 3 | 0 | Lig. : 4, Car. : 3 | TABS | CRLF | Windows

Résultats | Messages | Plan d'exécution


```
Requête 1 : coût de requête (par rapport au lot) : 100%
SELECT * FROM sys.dm_os_wait_stats ORDER BY wait_time_ms
```



Activer Windows
Accédez aux paramètres pour act

(17.0 RTM) | TechShop_DB | 00:00:00 | Ligne : 1, Col : 1 | 1 546 ligne

Comparaison des performances après optimisation

Après la création de l'index, la requête a été rejouée.
L'analyse du plan d'exécution montre désormais l'utilisation de l'index non clusterisé **IX_COMMANDES_ClientID**, ce qui réduit significativement le coût de la requête.

Les bénéfices observés :

- accès plus ciblé aux données,
- réduction des lectures inutiles,
- meilleure scalabilité de la requête.

Analyse des wait statistics

Les **wait statistics** ont été consultées via la vue système sys.dm_os_wait_stats afin d'identifier les principales causes d'attente sur l'instance SQL Server.

Les types d'attente majoritaires observés sont principalement liés à :

- des attentes système normales (scheduler, sémaphores),
- des opérations internes de SQL Server,
- des traitements de requêtes sans contention critique.

Aucune attente bloquante majeure (verrous excessifs ou I/O critiques) n'a été identifiée, ce qui indique que l'instance fonctionne correctement dans ce contexte de charge.